

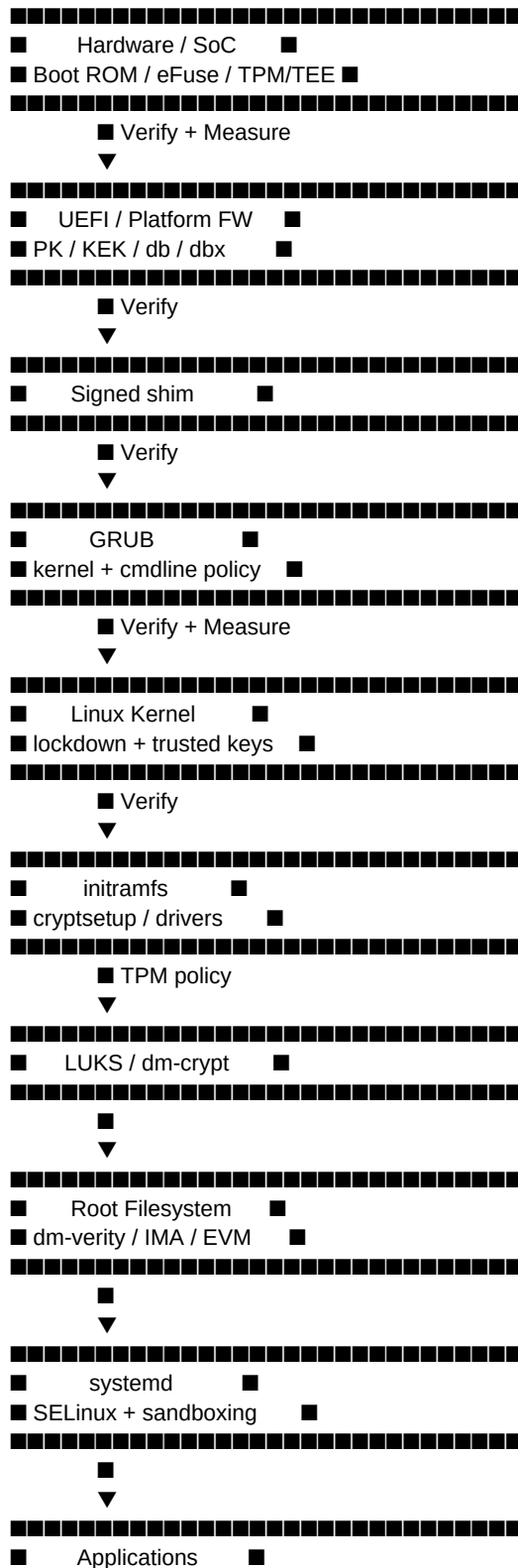
Secure Linux Boot Architecture

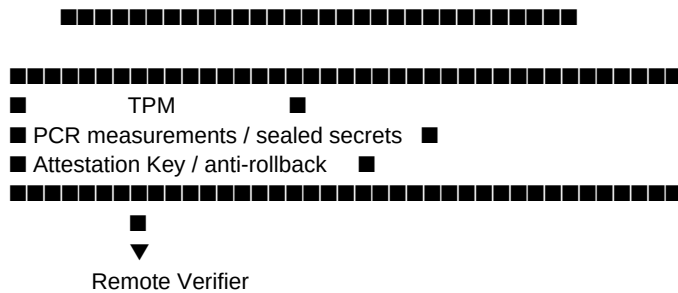
HLD + LLD + C++ Reference Code + Senior-Level Interview Answers

Target: Senior SDE / Kernel Engineer interview preparation.

Core principle: Root of Trust → Verify → Measure → Protect Keys → Encrypt Storage → Verify Root FS → Lock Down Kernel → Isolate Services → Attest → Securely Update.

1. HLD — High-Level Design





2. HLD — Component Responsibilities

Component	Responsibility
Hardware Root of Trust	Immutable trust anchor; verifies first executable stage.
UEFI Secure Boot	Authenticates EFI binaries using PK/KEK/db/dbx.
Shim	Distribution-trusted bridge from UEFI to GRUB.
GRUB	Authenticates and loads kernel/initramfs; protects boot policy.
Linux Kernel	Trusted keyrings, signed modules, lockdown, runtime security.
TPM 2.0	PCR measurements, sealed keys, attestation and anti-rollback support.
LUKS/dm-crypt	Confidentiality of data at rest.
dm-verity	Integrity of immutable root filesystem.
IMA/EVM	Measurement/appraisal of mutable files.
systemd/SELinux	Service isolation, least privilege and mandatory access control.
A/B Update	Atomic updates, health checks and rollback.

3. Secure Boot vs Measured Boot

Secure Boot:

- "Should I execute this?"
- > signature verification
- > prevent unauthorized code from running

Measured Boot:

- "What exactly did I execute?"
- > hash measurements
- > TPM PCR extension
- > evidence for attestation

TPM:

- protects keys + stores measurements + provides attestation

4. LLD — Core Interfaces

```
class BootArtifact {
public:
    virtual ~BootArtifact() = default;
    virtual std::string name() const = 0;
    virtual std::vector<uint8_t> digest() const = 0;
    virtual std::vector<uint8_t> signature() const = 0;
};

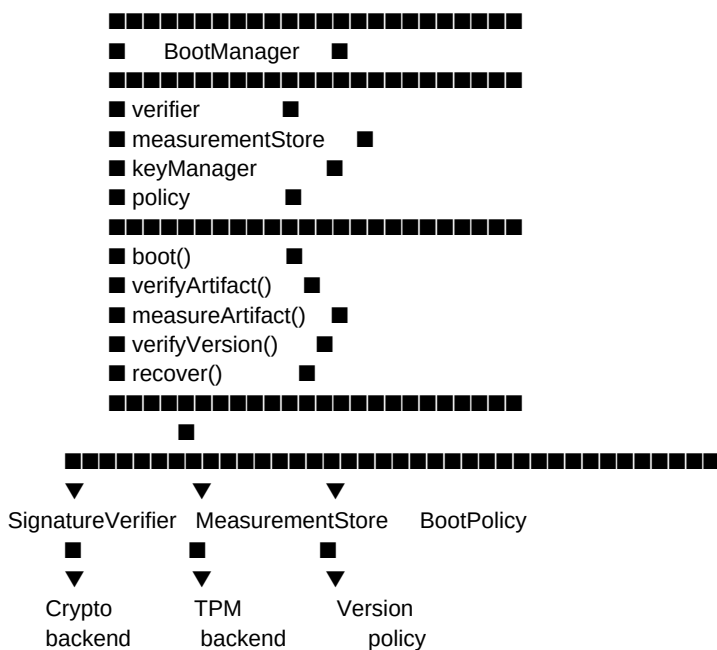
class SignatureVerifier {
public:
    virtual ~SignatureVerifier() = default;
    virtual bool verify(const BootArtifact& artifact) = 0;
};

class MeasurementStore {
public:
    virtual ~MeasurementStore() = default;
    virtual bool extendPCR(uint32_t pcr,
                          const std::vector<uint8_t>& digest) = 0;
};

class KeyManager {
public:
    virtual ~KeyManager() = default;
    virtual bool isTrusted(const std::vector<uint8_t>& keyId) = 0;
    virtual bool revoke(const std::vector<uint8_t>& keyId) = 0;
};

class BootPolicy {
public:
    virtual ~BootPolicy() = default;
    virtual bool allow(const BootArtifact& artifact) = 0;
    virtual bool antiRollback(uint64_t version) = 0;
};
```

5. LLD — Boot Manager



6. Detailed Boot Sequence

BootManager::boot()

```

|
+--> readActiveSlot()
+--> loadMetadata()
+--> verifyVersion()
+--> verifyArtifact(bootloader)
+--> measureArtifact(bootloader)
+--> verifyArtifact(kernel)
+--> verifyArtifact(initramfs)
+--> measureArtifact(kernel)
+--> measureArtifact(initramfs)
+--> verifyCommandLine()
+--> verifyTPMPolicy()
+--> unlockRootFS()
+--> verifyRootFS()
+--> markBootAttempt()
+--> handOffToKernel()

```

7. LLD — Data Structures

```

struct ArtifactMetadata {
    std::string name;
    uint64_t version;
    std::array<uint8_t, 32> sha256;
    std::vector<uint8_t> signature;
    std::string signingKeyId;
};

enum class BootSlot { A, B };

struct SlotMetadata {
    BootSlot slot;
    uint64_t version;
    bool bootable;
    bool successful;
    uint32_t attempts;
};

struct TPMPolicy {
    uint32_t pcrIndex;
    std::array<uint8_t, 32> expectedDigest;
    uint64_t minimumVersion;
};

struct BootResult {
    bool success;
    std::string error;
};

```

8. C++ Reference Implementation

This is interview-grade reference code. Production software should use established cryptographic and TPM libraries instead of implementing cryptographic primitives directly.

```
#include <array>
#include <cstdint>
#include <iostream>
#include <string>
#include <vector>

class SignatureVerifier {
public:
    bool verify(const std::string& image,
               const std::vector<uint8_t>& signature,
               const std::string& keyId)
    {
        // Production: hash image, verify signature,
        // validate certificate chain and revocation.
        std::cout << "Verifying " << image
                  << " using key " << keyId << "\n";
        return !image.empty() &&
               !signature.empty() &&
               !keyId.empty();
    }
};

class TPM {
public:
    bool extendPCR(uint32_t pcr,
                  const std::array<uint8_t, 32>& digest)
    {
        // Production: TPM2_PCR_Extend.
        std::cout << "Extending PCR " << pcr << "\n";
        return true;
    }

    bool authorizeUnlock(
        uint32_t pcr,
        const std::array<uint8_t, 32>& expected)
    {
        // Production: TPM2 policy session + sealed object.
        std::cout << "Checking TPM policy on PCR "
                  << pcr << "\n";
        return true;
    }
};

class RootFS {
public:
    bool unlock()
    {
        // Production: cryptsetup/LUKS or equivalent.
        std::cout << "Unlocking encrypted root filesystem\n";
        return true;
    }

    bool verify()
    {
        // Production: dm-verity or IMA/EVM.
        std::cout << "Verifying root filesystem\n";
        return true;
    }
};

class BootManager {
public:
```

```

BootManager(SignatureVerifier& verifier,
            TPM& tpm,
            RootFS& rootfs)
: verifier_(verifier), tpm_(tpm), rootfs_(rootfs) {}

```

```

bool boot()
{
    if (!verifyBootloader())
        return fail("bootloader verification failed");

    if (!verifyKernel())
        return fail("kernel verification failed");

    if (!verifyInitramfs())
        return fail("initramfs verification failed");

    if (!measureBoot())
        return fail("TPM measurement failed");

    if (!authorizeDiskUnlock())
        return fail("TPM authorization failed");

    if (!rootfs_.unlock())
        return fail("rootfs unlock failed");

    if (!rootfs_.verify())
        return fail("rootfs integrity verification failed");

    std::cout << "Secure boot successful\n";
    return true;
}

```

private:

```

bool verifyBootloader()
{
    return verifier_.verify(
        "grub.efi", {0x01, 0x02, 0x03},
        "BOOTLOADER_SIGNING_KEY");
}

```

```

bool verifyKernel()
{
    return verifier_.verify(
        "vmlinuz", {0x04, 0x05, 0x06},
        "KERNEL_SIGNING_KEY");
}

```

```

bool verifyInitramfs()
{
    return verifier_.verify(
        "initramfs.img", {0x07, 0x08, 0x09},
        "INITRAMFS_SIGNING_KEY");
}

```

```

bool measureBoot()
{
    std::array<uint8_t, 32> digest{};
    return tpm_.extendPCR(7, digest);
}

```

```

bool authorizeDiskUnlock()
{
    std::array<uint8_t, 32> expected{};
    return tpm_.authorizeUnlock(7, expected);
}

```

```

bool fail(const std::string& reason)

```

```

{
    std::cerr << "SECURE BOOT FAILURE: "
                << reason << "\n";
    return false;
}

SignatureVerifier& verifier_;
TPM& tpm_;
RootFS& rootfs_;
};

int main()
{
    SignatureVerifier verifier;
    TPM tpm;
    RootFS rootfs;

    BootManager bootManager(verifier, tpm, rootfs);
    return bootManager.boot() ? 0 : 1;
}

```

9. TPM Measurement Pseudocode

```

bool measureArtifact(TPM& tpm,
                    uint32_t pcr,
                    const std::vector<uint8_t>& image)
{
    auto digest = SHA256(image);

    // Conceptually:
    // PCR_new = HASH(PCR_old || digest)

    return tpm.extendPCR(pcr, digest);
}

```

10. Anti-Rollback Pseudocode

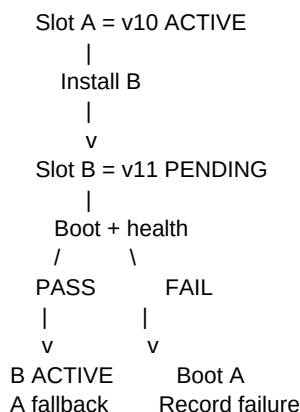
```

bool checkVersion(uint64_t imageVersion,
                 uint64_t minimumVersion)
{
    if (imageVersion < minimumVersion) {
        std::cerr << "Rollback detected\n";
        return false;
    }

    return true;
}

```

11. A/B Update State Machine



12. Senior-Level Interview Questions & Answers

Q1. What is the difference between Secure Boot and Measured Boot?

Answer: Secure Boot is preventive: it verifies a digital signature before executing a boot component. Measured Boot is evidentiary: it hashes boot components and extends those measurements into TPM PCRs. Secure Boot asks “Should I execute this?” while Measured Boot asks “What exactly did I execute?” Measured Boot is particularly useful for remote attestation.

Q2. Why is TPM needed if UEFI Secure Boot already exists?

Answer: UEFI Secure Boot protects the local boot process, but TPM adds measured boot, key sealing and remote attestation. A disk key can be released only when PCRs satisfy an approved policy. A remote verifier can also verify a TPM quote and determine whether the machine booted an approved software configuration.

Q3. How does PCR extend work?

Answer: A PCR is extended rather than simply overwritten. Conceptually $PCR_new = SHA256(PCR_old || \text{measurement})$. Each boot component changes the resulting PCR state, so modifying any measured component produces a different final value. This gives a tamper-evident representation of the boot history.

Q4. How would you prevent an attacker from booting an older signed kernel?

Answer: Signature verification alone is insufficient because an older vulnerable kernel can still be legitimately signed. I would implement anti-rollback protection: image signature must be valid AND image_version must be greater than or equal to a protected minimum version. The minimum version should be protected using TPM NV storage, hardware monotonic counters or secure firmware storage.

Q5. How would you protect the initramfs?

Answer: The initramfs executes before the normal root filesystem and therefore is part of the trusted computing base. I would authenticate it, measure it into TPM PCRs, protect the kernel command line, use signed modules, keep the initramfs minimal and use immutable/protected storage where practical.

Q6. How does Kernel Lockdown complement Secure Boot?

Answer: Secure Boot protects the system before the kernel executes. Kernel Lockdown helps preserve that trust boundary after Linux starts by restricting operations that could undermine kernel integrity, such as certain kernel memory access, debugging mechanisms and unauthorized module loading. I would combine it with signed modules, SELinux/AppArmor and least privilege.

Q7. How do signed kernel modules work?

Answer: A kernel module contains a digital signature. The kernel verifies that signature against its trusted keyrings before loading the module. With signature enforcement enabled, unsigned or untrusted modules are rejected. This prevents an attacker from easily loading arbitrary kernel-space code.

Q8. How would you design a secure A/B update system?

Answer: Maintain two bootable slots. If A is active, write the signed update to B, verify it, mark B pending and reboot. B performs health checks for the kernel, rootfs and critical services. On success, B becomes active. On failure or repeated watchdog resets, the bootloader returns to A. Metadata, signatures and version information must themselves be authenticated and protected against rollback.

Q9. What happens if the kernel signing key is compromised?

Answer: Treat it as a security incident. Revoke the compromised key, distribute a trusted replacement key, sign new kernels, update revocation databases such as UEFI dbx where applicable, and use secure recovery/update mechanisms. Signing keys should be separated by trust domain and sensitive root keys should remain offline or hardware protected.

Q10. How would a remote server verify that a Linux machine booted an approved image?

Answer: Use TPM remote attestation. Boot components are measured into PCRs. The verifier sends a nonce and the TPM returns a signed quote containing PCR values and the nonce. The verifier checks the TPM quote signature, nonce and PCR values against an approved boot policy. If they match, the machine can be admitted to the trusted environment.

Q11. Why use dm-verity instead of a normal filesystem hash?

Answer: A single filesystem hash is not practical for efficient runtime block verification. dm-verity uses a Merkle hash tree, allowing individual blocks to be verified as they are read. If a block is modified, its hash no longer matches the tree and the I/O can be rejected. This is especially useful for immutable Linux root filesystems.

Q12. How would you securely recover from a failed kernel update?

Answer: Use authenticated A/B slots, a signed recovery image, watchdogs, boot-attempt counters, health checks and anti-rollback. A failed new slot is marked bad and the previous known-good slot is booted. Recovery must remain inside the same chain of trust; a boot failure must not become a reason to bypass Secure Boot.

Q13. What happens if an attacker has physical access to the machine?

Answer: Define the physical attacker model explicitly. Secure Boot protects the boot chain, TPM protects secrets and measurements, and disk encryption protects data at rest. Stronger physical protections may require IOMMU, firmware protections, tamper-resistant hardware and platform security controls. No software-only architecture can guarantee security against an unrestricted physical attacker.

13. 30-Second Answer to Memorize

Question: “Design a secure Linux boot architecture.”

Memorize this: I would build a chain of trust starting from an immutable hardware root of trust and UEFI Secure Boot. Firmware authenticates shim, shim authenticates GRUB, and GRUB authenticates the kernel and initramfs. In parallel, I would measure the boot components into TPM PCRs. The TPM can release LUKS encryption keys only when the measured state satisfies an approved policy. For filesystem integrity I'd use dm-verity for immutable rootfs or IMA/EVM for mutable files. Once Linux starts, kernel lockdown and signed modules protect the kernel boundary, while SELinux and systemd sandboxing protect user space. For updates I'd use signed A/B images with anti-rollback and health-based rollback. Finally, TPM remote attestation allows a remote verifier to establish that the machine booted an approved software stack.

14. Fast Revision Sheet

Concept	Remember
Secure Boot	Prevents unauthorized boot code from executing.
Measured Boot	Records boot measurements into TPM PCRs.
TPM	Protects keys, PCR measurements and attestation.
PCR Extend	$\text{PCR_new} = \text{HASH}(\text{PCR_old} \text{measurement})$.
LUKS	Encrypts data at rest.
dm-verity	Integrity verification for immutable rootfs.
IMA/EVM	Runtime file measurement/appraisal.
Lockdown	Protects kernel integrity after boot.
Module Signing	Prevents unauthorized kernel modules.
A/B Update	Atomic update + fallback.
Anti-rollback	Rejects older vulnerable signed images.
Remote Attestation	Remote proof of measured boot state.
Key Rotation	Limits impact of signing-key compromise.

15. Interview Strategy

For a Senior Kernel Engineer interview, do not stop at naming technologies. Explain the trust boundary, attacker model, why each mechanism exists, failure behavior, key compromise, rollback, recovery and operational trade-offs. A strong answer moves from HLD to LLD and then to concrete Linux mechanisms.